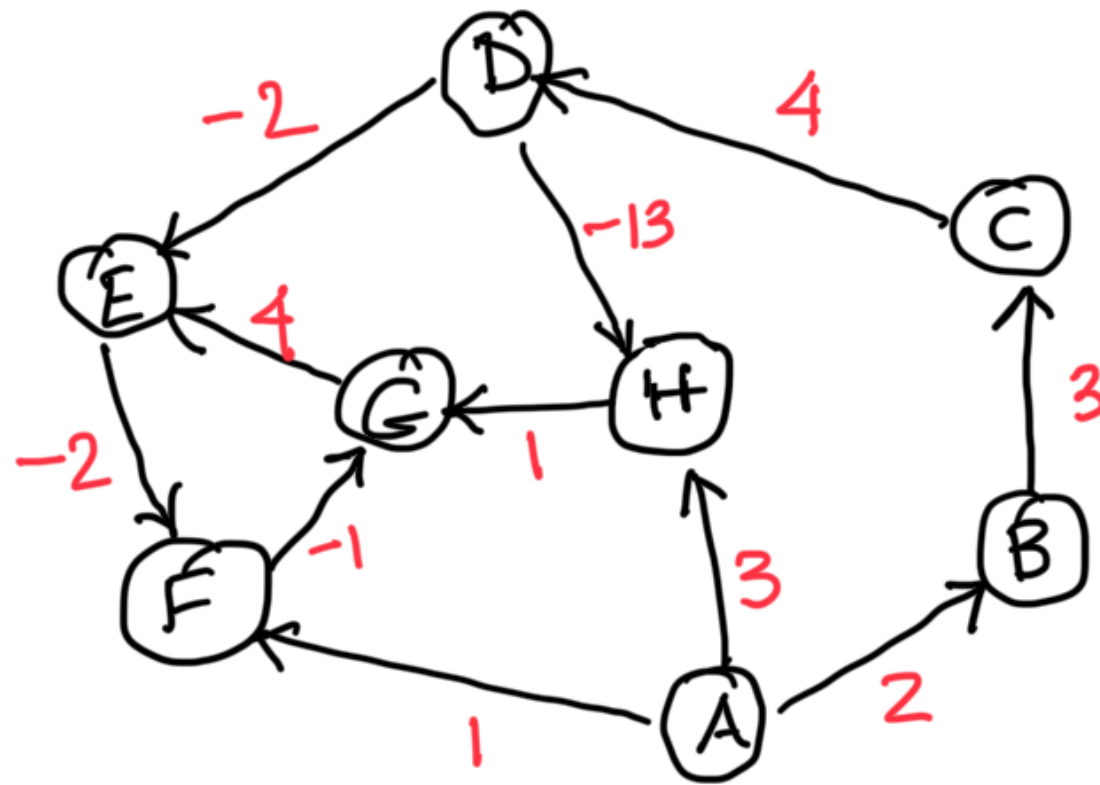




Terminating condition for Bellman Ford

Claim If $\Delta(v)$ decreases in the n^{th} iteration
 $\Leftrightarrow$ negative cycle

Proof Consider any negative cycle

$$W(v_1, v_2) + W(v_2, v_3) + \dots + W(v_k, v_1) = -\alpha < 0$$



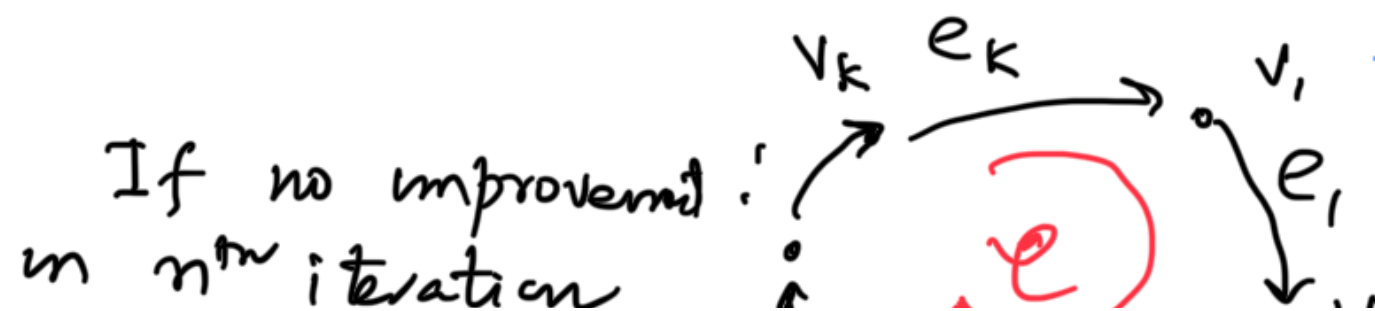
Relaxation sequence over consecutive iterations

$Relax(v_1, v_2), Relax(v_2, v_3) \dots Relax(v_k, v_1)$ will
result in $\Delta(v_1)$ being lowered by
at least $-\alpha$
after k iterations

$\Rightarrow \Delta(v_2)$ getting lowered by $-\alpha$ after $k+1$

$\Rightarrow \Delta(v_3)$ " " etc. $\Rightarrow$ Every successive
iteration, some $\Delta(v_i)$ will be lowered including n^{th} iter.

Terminating condition for BF is run for
 n iterations. If no improvement for any $\Delta(v)$
then no -ve cycle



Consider any
cycle C

$$\Delta^n(v_i) = \Delta^{n-1}(v_i)$$

$$\Delta^n(v_i) \leq \Delta^{n-1}(v_{i-1}) + \omega(v_{i-1}, v_i)$$

no relaxation)

$$\Delta^n(v_k) \leq \Delta^{n-1}(v_{k-1}) + \omega(v_{k-1}, v_k)$$

$$\Delta(v_{k-1}) \leq \Delta(v_{k-2}) + \omega(v_{k-2}, v_{k-1})$$

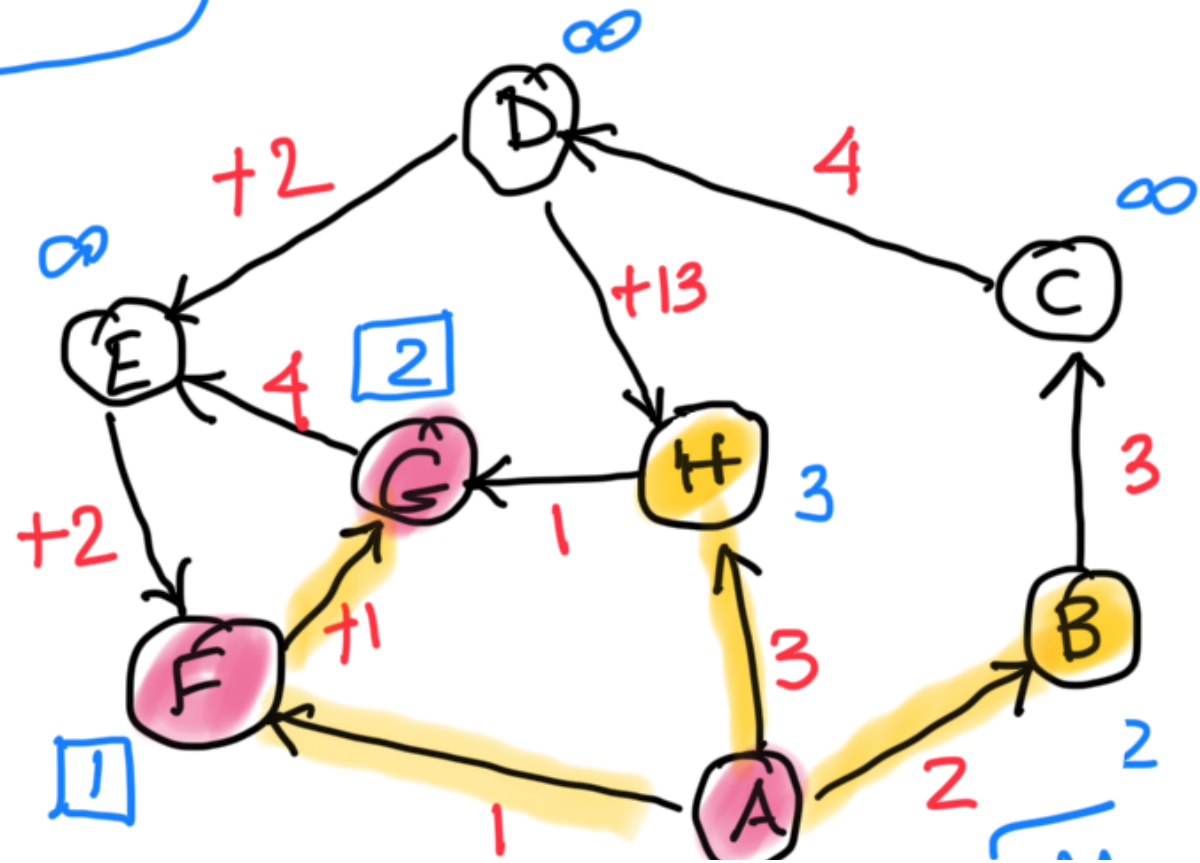
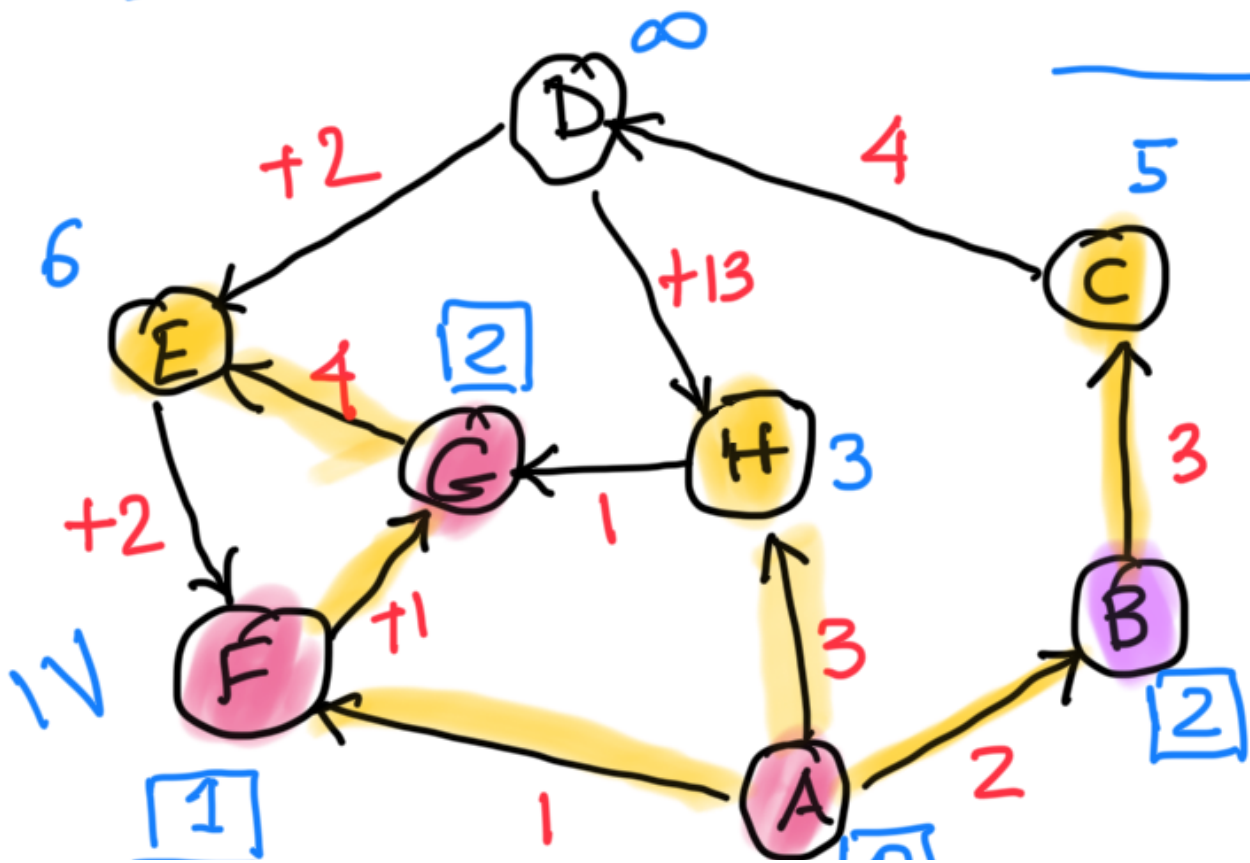
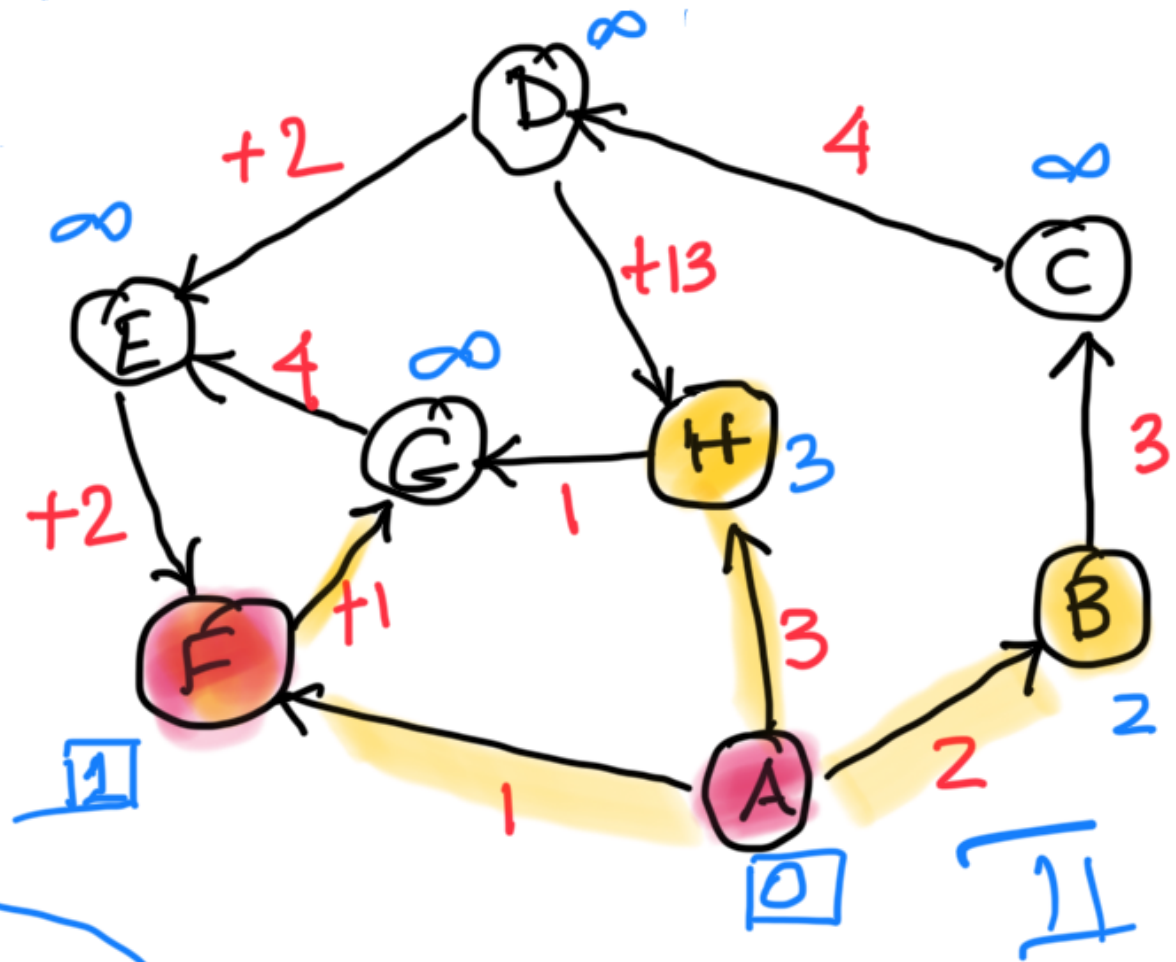
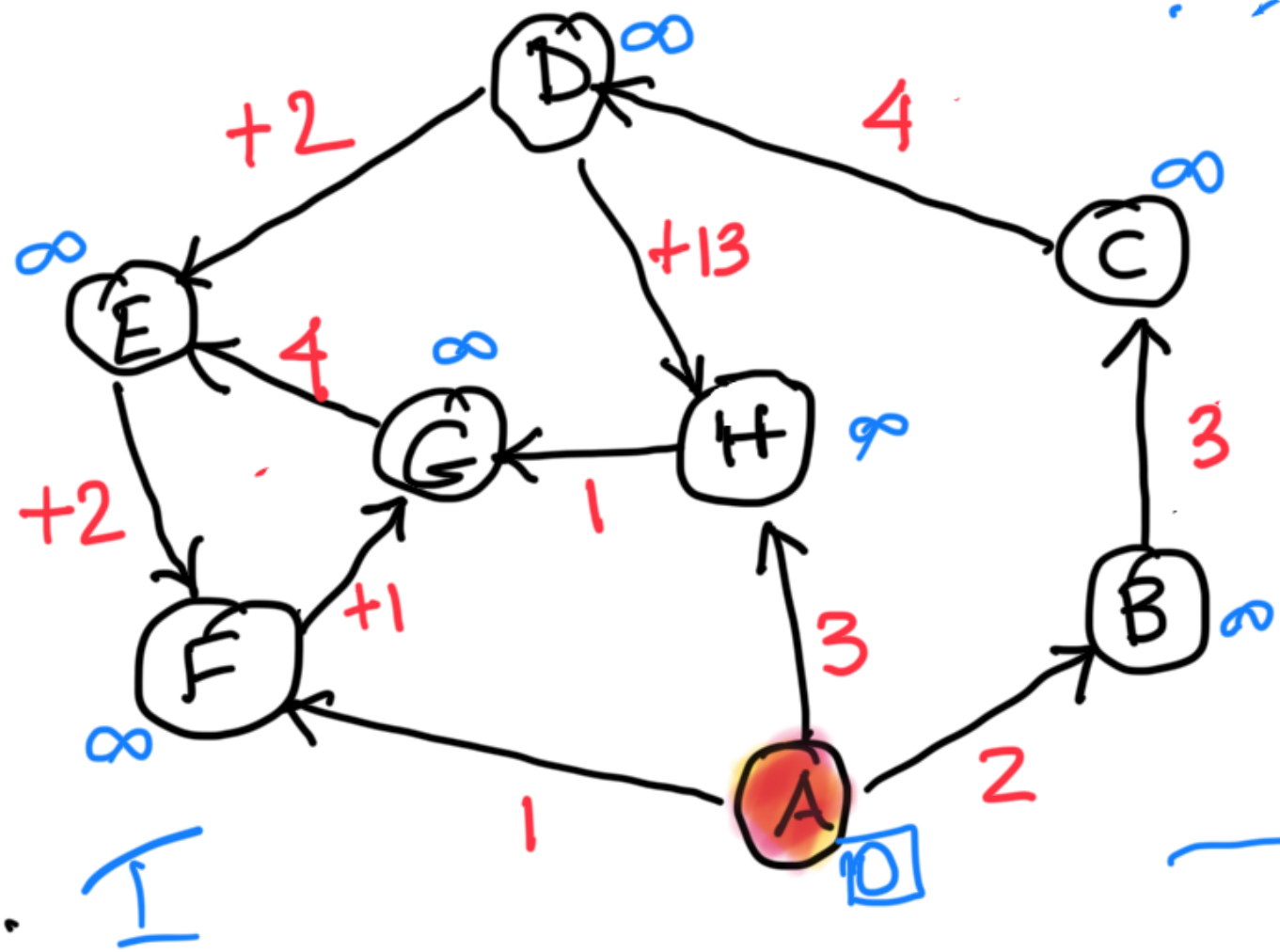
⋮

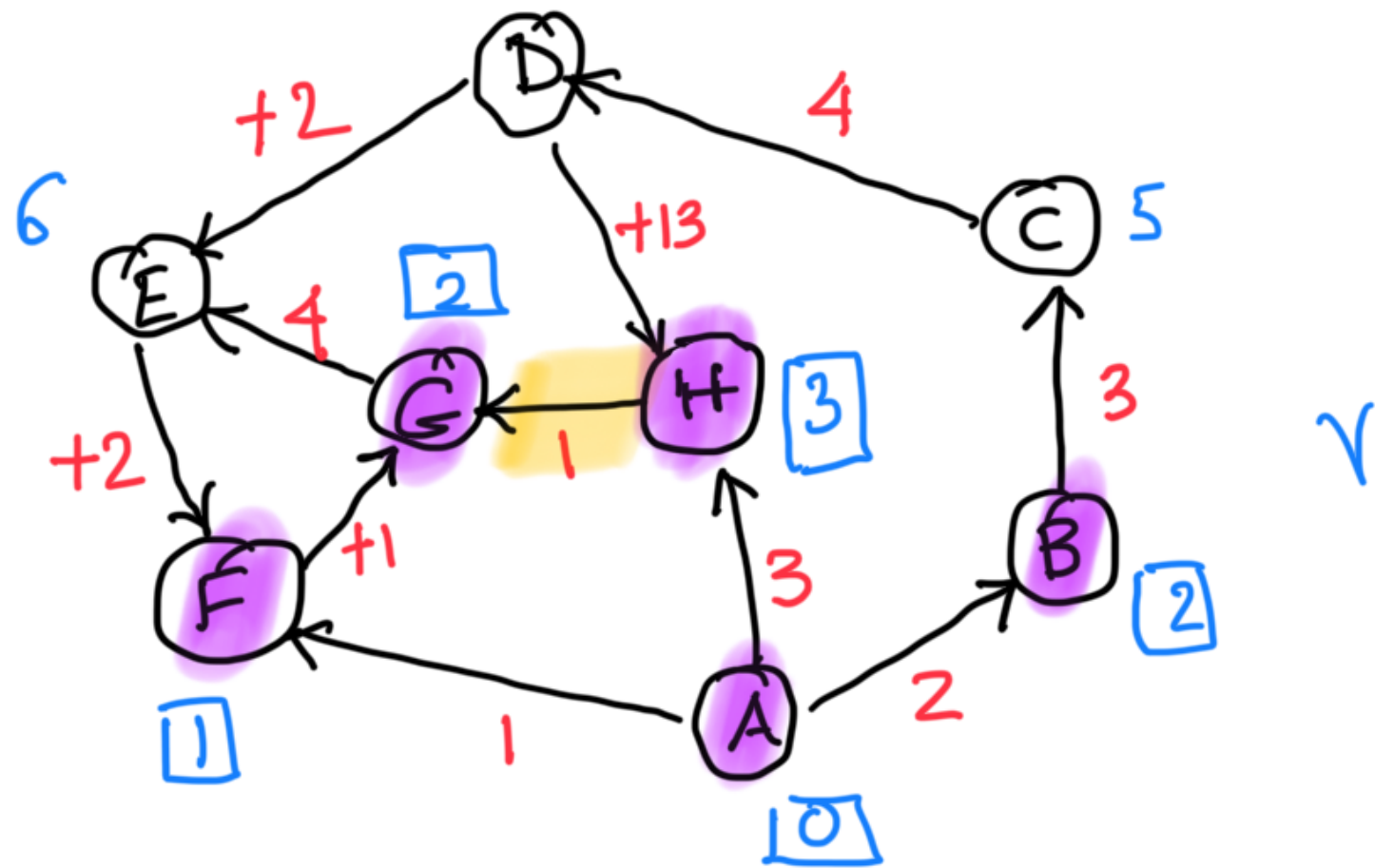
$$\Delta^n(v_1) \leq \Delta^{n-1}(v_k) + \omega(v_k, v_1)$$

Adding $\sum \cancel{\Delta^n(v_i)} \leq \sum \cancel{\Delta^{n-1}(v_i)} + \omega(\mathcal{C})$

$$v_2 = v_1, v_2, \dots, v_k$$

"NO NEGATIVE WEIGHTS!"





Dijkstra's Algorithm SSSP (non-negative weights)

V : Set of vertices

U : set of active vertices

S : set of processed vertices

U : set of frozen vertices

Initialize $S = \{s\}$ $U = V - \{s\}$

$$\Delta(s) = \delta(s) = 0 \quad \Delta(v) = \infty \quad v \in V - \{s\}$$

$$\text{latest} = s$$

while $U \neq \emptyset$ repeat

[Relax (latest, w) $\forall w \in \text{neighbors of latest}$
latest := $\underset{v \in U}{\operatorname{argmin}} \Delta(v)$
 $\delta(\text{latest}) = \Delta(\text{latest})$
 $S := S \cup \{\text{latest}\}$ latest is frozen]

Output $\delta(v)$ of $v \in S$

Running Time : $O((|E| + |V|) \log |V|)$
using min heaps

Correctness :

Claim : $\forall w$, when w is moved into S from U

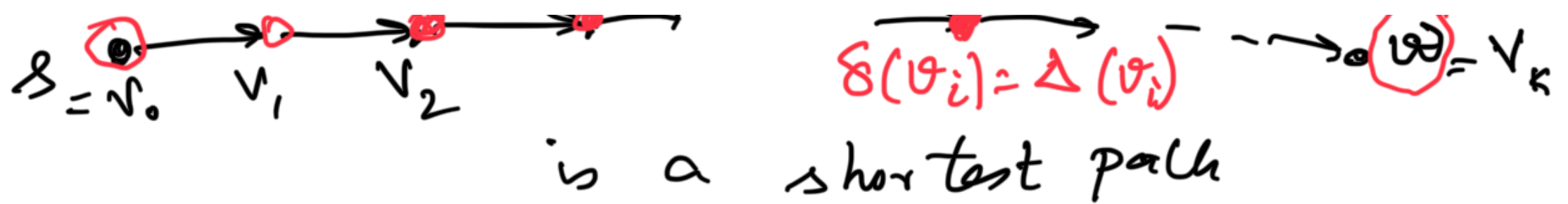
$$\delta(w) = \Delta(w)$$

Proof by contradiction : Suppose in a run of the algorithm, w is the first vertex for which $\Delta(w) \neq \delta(w)$ and $\Delta(w)$ is minimum

All the previously frozen vertices have $\delta(v_i) = \Delta(v_i)$

frozen

v_i = further from S



① $\Delta(v_k) > \delta(v_k)$ when it was frozen

when v_i was frozen (v_i, v_{i+1}) would be relaxed in the subsequent iteration
 ← this was correct

$$\Delta(v_{i+1}) = \delta(v_i) + w(v_i, v_{i+1})$$

$$= \delta(v_{i+1}) \quad (\text{since it is on the shortest path to } w)$$

$$\leq \delta(v_k) \quad \text{non negative wts}$$

So $\Delta(v_k) > \delta(v_k) \geq \Delta(v_{i+1})$ and v_k shouldn't have been chosen

Contradiction that $\Delta(w)$ was maximum

Useful property of Dijkstra's algorithm

The vertices are added to S (frozen)
in order of increasing distances from the
starting vertex